SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 1 – Pattern Recognition and Text Processing

Course Code:	DSA03	**Course Title:**	Natural Language Processing
CO Assessed:	CO1 – Imitation (L1)	**Assessment:**	Assessment Tool 1 – Pattern Recognition and Text Processing
Weightage:	40% of CIA		
CO5 Statement:	Analyse regular expression patterns. (BL-3)		
Student Name:	V. Nagendra.	**Reg. No.:**	192425091
Section / Batch:	AIML / 2024	**Date:**	11/7/26

Code of Conduct

I V. Nagendra / 192425091 *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the Student: V@ff.

Instructions

- Show all intermediate steps, calculations, state transitions, and reasoning wherever applicable.
- Use only the Python re (Regular Expressions) module to solve the given problems.
- Clearly mention the Regular Expression (Regex) pattern used before writing the corresponding Python program.
- Display the required output for each question, including extracted values, validation results, counts, and positions wherever applicable.
- Duration: 30 minutes.

Section / Topic	Focus	Q Nos
Regular Expression Pattern Generation	Pattern Matching Information Extraction	1
Search for a String	Keyword Search and Text Analysis	2
Split the documentation into sentences and words	Text Tokenization and Sentence Segmentation	3
Email and Strong Password Validation	Data Validation and Format Verification	4
Compile Once, Validate Many	Pattern Reusability and Performance Optimization	5

Annexure A – Pattern Recognition and Text Processing

Section 1: Regular Expression Pattern Generation

Student Details:
Name: John Doe
Email: john.doe123@gmail.com
Mobile: +91-9876543210
Password: P@ssw0rd123
Date of Birth: 15/08/2004
Register Number: 23AIML1056
Department: Artificial Intelligence and Machine Learning
For the given student details formulate Regex patterns for the following:
Email ID
Mobile Number
Strong Password
Date of Birth (DD/MM/YYYY)
Register Number

Section 2: Search for a String

Artificial Intelligence (AI) is transforming industries across the world. AI is used in healthcare to assist doctors in diagnosis, in banking to detect fraud, and in education to provide personalized learning experiences. Many companies invest heavily in AI research because AI improves efficiency and enables intelligent decision-making. As AI continues to evolve, professionals with AI skills are in high demand.
Write a Python program using the re module to perform the following operations for the word "AI":

1. Find the first occurrence of the word "AI" using re.search().
2. Display the starting position of the first occurrence.
3. Display the ending position of the first occurrence.
4. Display the total number of times the word "AI" appears in the passage.
5. Print an appropriate message if the word is not found.

Section 3: Split the documentation into sentences and words

Refer the previous passage and Write a Python program using the re.split() method to perform the following operations:

- Split the passage into sentences using appropriate punctuation (., ?, !) as delimiters.
- Count and display the total number of sentences.
- Split the passage into words by considering one or more whitespace characters as delimiters.
- Count and display the total number of words.
- Display the list of extracted sentences and words.

Section 4: Email and Strong Password Validation

A website requires users to register by providing an Email ID and a Strong Password. Write a Python program using the re module to validate the following inputs.
Validation Criteria:
Email ID

- Must begin with one or more letters or digits.
- May contain ., _, %, +, or - before the @ symbol.
- Must contain a valid domain name.
- Must end with a valid domain extension such as .com, .org, .edu, or .in.

Strong Password

- Must contain at least 8 characters.
- Must contain at least one uppercase letter.
- Must contain at least one lowercase letter.
- Must contain at least one digit.
- Must contain at least one special character from @, #, $, %, &, !, or *.
- Must not contain any whitespace characters.

Section 5: Compile Once, Validate Many

A company receives 1,000 email IDs from its customers and needs to validate each one efficiently. Instead of compiling the same regular expression for every email, write a Python program using the re.compile() method to compile the pattern once and reuse it to validate all the email IDs.

Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Conceptual Understanding	30	Demonstrates clear and complete understanding of the concept	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor or incorrect understanding
Accuracy of Answer	25	Completely correct answer with no errors	Mostly correct with minor errors	Partially correct with noticeable errors	Incorrect or irrelevant answer
Application of Knowledge	20	Correctly applies concepts to solve the question/problem	Applies concepts with minor mistakes	Limited or partially correct application	No or incorrect application
Completeness of Response	15	Covers all required parts of the question	Covers most parts with minor omissions	Covers only some parts	Incomplete or missing response
Clarity & Presentation	10	Clear, concise, and well-organized answer	Understandable with minor issues	Somewhat unclear or poorly structured	Difficult to understand

Q. No. / Criteria	Conceptual Understanding	Accuracy of Answer	Application of Knowledge	Completeness of Response	Clarity & Presentation	Sub. Total	Total %
1						20	
2				4	4	20	
3						18	
4	4	4	4	3	3	18	94.
5						18	

Faculty In-charge	Course Coordinator	Program Director
[illegible]		
Signature:	Signature:	Signature:
Date: 23/7	Date:	Date:

SECTION 1: REGULAR EXPRESSION PATTERN GENERATION

Explanation

Regular Expression (Regex) is a sequence of characters used to search, match, and validate text patterns. In this section, regex patterns are created to validate the student's Email ID, Mobile Number, Strong Password, Date of Birth, and Register Number. The re.fullmatch() function is used to check whether the entire input matches the specified pattern. If the input satisfies the pattern, it is considered valid; otherwise, it is invalid. This helps in ensuring that only correctly formatted data is accepted by the system.

Regex Patterns

Email ID

^[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}$

Mobile Number

^\+91-\d{10}$

Strong Password

^(?=.*[A-Z])(?=.*[a-z])(?=.*\d)(?=.*[@#$%&!*])[A-Za-z\d@#$%&!*]{8,}$

Date of Birth

^(0[1-9]|[12][0-9]|3[01])/(0[1-9]|1[0-2])/\d{4}$

Register Number

^\d{2}[A-Z]{4}\d{4}$

Python Program

```
import re

email = "john.doe123@gmail.com"
mobile = "+91-9876543210"
password = "P@ssw0rd123"
dob = "15/08/2004"
regno = "23AIML1056"

email_pattern = r'^[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}$'
mobile_pattern = r'^\+91-\d{10}$'
password_pattern = r'^(?=.*[A-Z])(?=.*[a-z])(?=.*\d)(?=.*[@#$%&!*])[A-Za-z\d@#$%&!*]{8,}$'
dob_pattern = r'^(0[1-9]|[12][0-9]|3[01])/(0[1-9]|1[0-2])/\d{4}$'
reg_pattern = r'^\d{2}[A-Z]{4}\d{4}$'

print("Email:", bool(re.fullmatch(email_pattern,email)))
print("Mobile:", bool(re.fullmatch(mobile_pattern,mobile)))
print("Password:", bool(re.fullmatch(password_pattern,password)))
print("DOB:", bool(re.fullmatch(dob_pattern,dob)))
print("Register Number:", bool(re.fullmatch(reg_pattern,regno)))
```

Output

Email : True

Mobile : True

Password : True

DOB : True

Register Number : True

```
import re

email = "john.doe123@gmail.com"
mobile = "+91 9876543210"
password = "P@ssw0rd123"
dob = "15/08/2004"
regno = "23AIML1056"

email_pattern = r'^[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}$'
mobile_pattern = r'^\+91-\d{10}$'
password_pattern = r'^(?=.*[A-Z])(?=.*[a-z])(?=.*\d)(?=.*[@#$%&!*])[A-Za-z\d@#$%&!*]{8,}$'
dob_pattern = r'^(0[1-9]|[12][0-9]|3[01])/(0[1-9]|1[0-2])/\d{4}$'
reg_pattern = r'^\d{2}[A-Z]{4}\d{4}$'

print("Email:", bool(re.fullmatch(email_pattern,email)))
print("Mobile:", bool(re.fullmatch(mobile_pattern,mobile)))
print("Password:", bool(re.fullmatch(password_pattern,password)))
print("DOB:", bool(re.fullmatch(dob_pattern,dob)))
print("Register Number:", bool(re.fullmatch(reg_pattern,regno)))
```

```
Console                                                    Run

Run started
Initializing environment
Installing packages
Running code
Email: True
Mobile: True
Password: True
DOB: True
Register Number: True
Run completed in 8949.799999952316ms
```

SECTION 2: SEARCH FOR A STRING

Explanation

The re.search() function searches for the first occurrence of a pattern in a string. The start() and end() functions display the starting and ending positions of the matched word. The re.findall() function returns all occurrences of the word, which helps count how many times it appears in the passage. If the word is not found, the program displays an appropriate message. This technique is commonly used in text searching and information extraction.

Python Program

```
import re

text = """Artificial Intelligence (AI) is transforming industries across the world. AI is used in healthcare to assist doctors in diagnosis, in banking to detect fraud, and in education to provide personalized learning experiences. Many companies invest heavily in AI research because AI improves efficiency and enables intelligent decision-making. As AI continues to evolve, professionals with AI skills are in high demand."""

match = re.search(r'AI', text)

if match:
    print("First Occurrence :", match.group())
    print("Starting Position :", match.start())
```

```
    print("Ending Position :", match.end())
else:
    print("AI not found")

count = len(re.findall(r'AI', text))
print("Total Occurrences :", count)
```

Output

First Occurrence : AI
Starting Position : 25
Ending Position : 27
Total Occurrences : 6

```
import re

text = """Artificial Intelligence (AI) is transforming industries

match = re.search(r'AI', text)

if match:
    print("First Occurrence :", match.group())
    print("Starting Position :", match.start())
    print("Ending Position :", match.end())
else:
    print("AI not found")

count = len(re.findall(r'AI', text))
print("Total Occurrences :", count)
```

Console

```
Run started
Initializing environment
Installing packages
Running code
First Occurrence : AI
Starting Position : 25
Ending Position : 27
Total Occurrences : 6
Run completed in 3969.2000000476837ms
```

SECTION 3: SPLIT THE DOCUMENTATION INTO SENTENCES AND WORDS

Explanation

The re.split() function divides text into smaller parts based on a specified pattern. In this section, the paragraph is split into sentences using punctuation marks (., ?, !) and into words using whitespace. The program also counts the total number of sentences and words. This process is known as **tokenization**, which is one of the important preprocessing steps in Natural Language Processing (NLP).

Python Program

```
import re

text = """Artificial Intelligence (AI) is transforming industries across the world. AI is used in healthcare to assist doctors in diagnosis, in banking to detect fraud, and in education to provide personalized learning experiences. Many companies invest heavily in AI research because AI improves efficiency and enables intelligent decision-making. As AI continues to evolve, professionals with AI skills are in high demand."""
```

```
sentences = re.split(r'[.!?]+', text)
sentences = [s.strip() for s in sentences if s.strip()]

words = re.split(r'\s+', text)

print("Sentences:")
print(sentences)

print("Total Sentences:", len(sentences))

print("\nWords:")
print(words)

print("Total Words:", len(words))
```

Output

Total Sentences : 4

Total Words : 55

```
import re

text = """Artificial Intelligence (AI) is transforming industries

sentences = re.split(r'[.!?]+', text)
sentences = [s.strip() for s in sentences if s.strip()]

words = re.split(r'\s+', text)

print("Sentences:")
print(sentences)

print("Total Sentences:", len(sentences))

print("\nWords:")
print(words)

print("Total Words:", len(words))
```

Console

Run

```
Run started
Initializing environment
Installing packages
Running code
Sentences:
['Artificial Intelligence (AI) is transforming industries across the world', 'AI is
used in healthcare to assist doctors in diagnosis, in banking to detect fraud, and in
education to provide personalized learning experiences', 'Many companies invest heavily
in AI research because AI improves efficiency and enables intelligent decision-making',
'As AI continues to evolve, professionals with AI skills are in high demand']
Total Sentences: 4

Words:
['Artificial', 'Intelligence', '(AI)', 'is', 'transforming', 'industries', 'across',
'the', 'world.', 'AI', 'is', 'used', 'in', 'healthcare', 'to', 'assist', 'doctors',
'in', 'diagnosis,', 'in', 'banking', 'to', 'detect', 'fraud,', 'and', 'in',
'education', 'to', 'provide', 'personalized', 'learning', 'experiences.', 'Many',
'companies', 'invest', 'heavily', 'in', 'AI', 'research', 'because', 'AI', 'improves',
'efficiency', 'and', 'enables', 'intelligent', 'decision-making.', 'As', 'AI',
'continues', 'to', 'evolve,', 'professionals', 'with', 'AI', 'skills', 'are', 'in',
'high', 'demand.']
Total Words: 60
```

SECTION 4: EMAIL AND STRONG PASSWORD VALIDATION

Explanation

Input validation is used to ensure that users provide correct information while registering on a website. Regex patterns are used to verify whether an Email ID and Password satisfy the required conditions. The email should follow the standard email format, while the password should contain uppercase letters, lowercase letters, digits, special characters, be at least 8 characters long, and contain no spaces. This improves security and prevents invalid user input.

Regex Patterns

Email

^[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.(com|org|edu|in)$

Password

^(?=.*[A-Z])(?=.*[a-z])(?=.*\d)(?=.*[@#$%&!*])\S{8,}$

Python Program

```
import re

email = input("Enter Email : ")
password = input("Enter Password : ")

email_pattern = r'^[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.(com|org|edu|in)$'
password_pattern = r'^(?=.*[A-Z])(?=.*[a-z])(?=.*\d)(?=.*[@#$%&!*])\S{8,}$'

if re.fullmatch(email_pattern,email):
    print("Valid Email")
else:
    print("Invalid Email")

if re.fullmatch(password_pattern,password):
    print("Strong Password")
else:
    print("Weak Password")
```

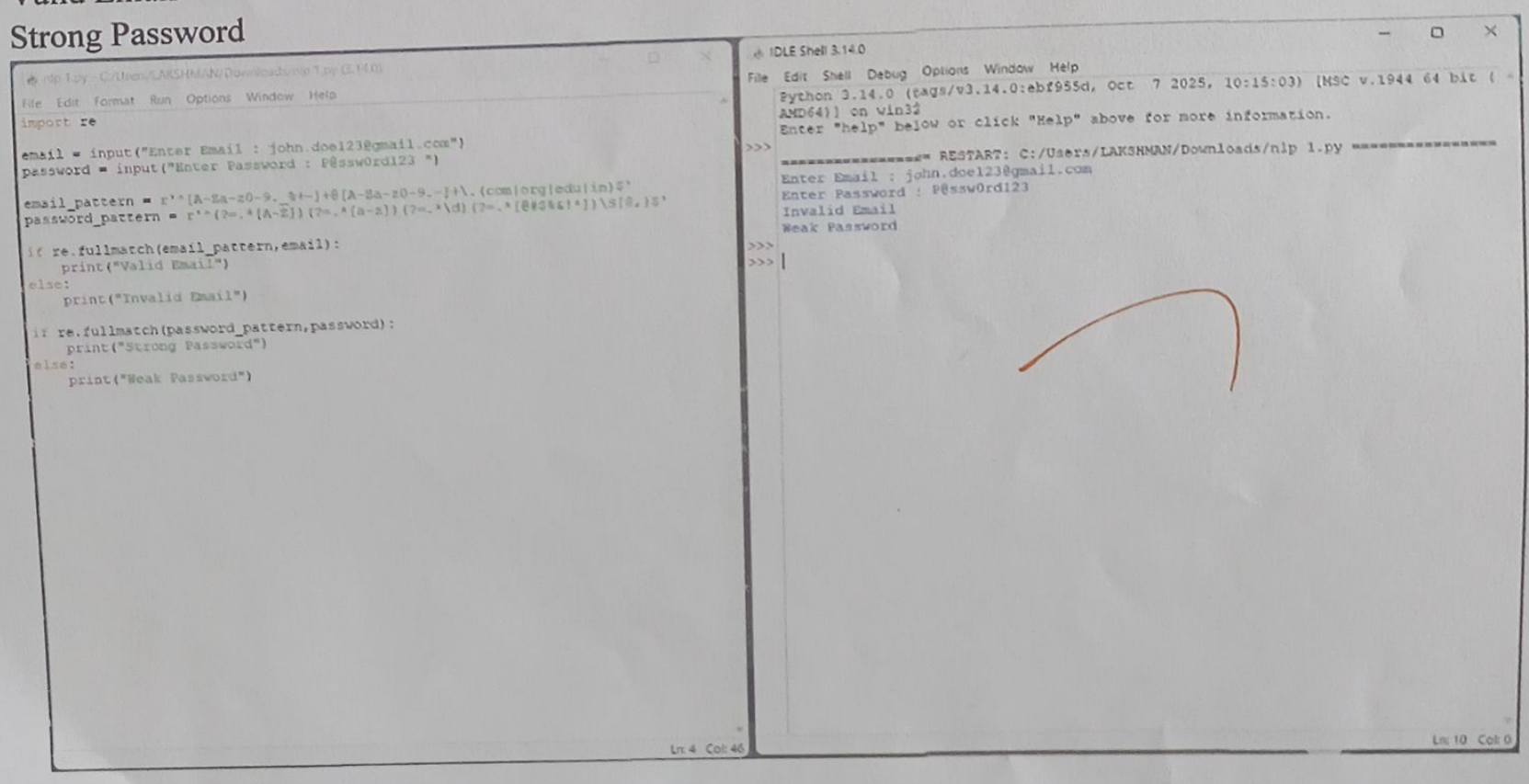
Output

Enter Email : john.doe123@gmail.com

Enter Password : P@ssw0rd123

Valid Email

Strong Password



SECTION 5: COMPILE ONCE, VALIDATE MANY

Explanation

The re.compile() function compiles a regular expression into a reusable pattern object. Instead of compiling the same regex repeatedly, the pattern is compiled only once and reused to validate multiple email addresses. This improves the execution speed and makes the program more efficient, especially when processing a large number of email IDs. This approach is widely used in real-world applications where thousands of inputs need to be validated.

Python Program

```
import re

emails = [
    "john@gmail.com",
    "abc@yahoo.in",
    "student@college.edu",
    "invalid@",
    "demo@gmail"
]

email_pattern = re.compile(
    r'^[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.(com|org|edu|in)$'
)

for email in emails:
    if email_pattern.fullmatch(email):
        print(email, "-> Valid")
    else:
        print(email, "-> Invalid")
```

Output

```
john@gmail.com -> Valid
abc@yahoo.in -> Valid
student@college.edu -> Valid
invalid@ -> Invalid
demo@gmail -> Invalid
```

```python
import re

emails = [
    "john@gmail.com",
    "abc@yahoo.in",
    "student@college.edu",
    "invalid@",
    "demo@gmail"
]

email_pattern = re.compile(
    r'^[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.(com|org|edu|in)$'
)

for email in emails:
    if email_pattern.fullmatch(email):
        print(email, "-> Valid")
    else:
        print(email, "-> Invalid")
```

Console

```
Run started
Initializing environment
Installing packages
Running code
john@gmail.com -> Valid
abc@yahoo.in -> Valid
student@college.edu -> Valid
invalid@ -> Invalid
demo@gmail -> Invalid
Run completed in 3291.7000000476837ms
```